

Here's the third puzzle.



You are given four cards, with each card having a digit on one side and an alphabet on the other.

The hypothesis you need to validate is that if a card has a vowel on one side, then it has an even number on the other side. The four cards are laid such that Card 1 shows A, Card 2 shows D, Card 3 shows 4 and Card 4 shows 7. You need to specify which of these four cards are needed to check whether this rule is true or not.

Most people give the answer as the cards containing A and 4. But this is incorrect. Can you figure out why? The correct answer is the cards containing A and 7. It is obvious why the 'A' card needs to be checked. If Card 'A' has an odd number on the other side, obviously the rule is incorrect. Why is it that Card '4' need not be checked? The reason is that if Card 4 contains a consonant on the other side, it still does not violate the rule. (If you disagree, think carefully about the rule that we are trying to verify here.) On the other hand, if Card 7 contains a vowel on its other side, then the rule is disproved. This is another example of how the short-cut heuristics employed by our brain to quickly jump to a solution can lead us to the wrong answer.

You may have been wondering why we are discussing cognitive intelligence puzzles in a column devoted for computer science. Computer science programming projects are littered with evidence where a poor choice or decision in the planning, requirements, implementation and testing phases has led to a costly debacle. Hence, it is all the more important for computer scientists to be able to reason effectively and be aware of the short-cut heuristics and illusions the human brain can come up with. I would like to suggest that our readers look up the book 'Thinking Fast and Slow' by Daniel Kahneman, a Nobel Prize laureate. The book is an effective guide to the various heuristics and biases the human mind is prone to, and on how to become an effective decision maker.

Vivekanandan, one of our student readers, had asked me about libraries for AI and NLP algorithms. Given that we also had a few other readers asking about how to ramp up on NLTK (Natural Language Toolkit), I felt it would be good to have a short discussion on the Python libraries available for AI and NLP and how one can quickly ramp up on the same. This month's article is being co-authored by one of our student readers, Ankith Subramanya from The International School, Bengaluru, as he shares his experience of ramping up on Python libraries for AI and NLP.

As computer science develops, it is exploring problems that arise in the real world. Artificial Intelligence (AI) and Natural Language Processing

(NLP) are two fields that have great practical applications. AI is the science of making our machines 'intelligent', so that they can be utilised for a number of practical problems. AI has vast potential and can have a great impact on every aspect of our lives, from the household (robotic vacuum cleaners) to national defence (missiles). As AI is a vast topic, research in it involves several centralised fields such as reasoning, knowledge, planning and learning. One of these several fields is NLP, which is probably the most human-centric field in AI, in the sense that it involves the interaction between natural human languages and the computer. This can be broadly understood as human-computer interactions. The goal of research in this field is to make communication between humans and computers as natural and effortless as possible, as if a computer were a person.

Python is regarded by many as a simple yet powerful language. The reason for its simplicity and quickness can be attributed to features such as built-in high level data types and dynamic typing. Python is increasingly preferred for AI, NLP and data mining. Apart from the characteristic benefits that it offers to developers, the main reason for Python's increasing preference is the vast number of tools and libraries that it offers. These can be broadly classified into general AI, machine learning, natural language and text processing, and neural networks. AMAI (Annals of Mathematics and Artificial Intelligence) is a library that includes a number of Python implementations of the algorithms from Russell and Norvig's 'Artificial Intelligence: A Modern Approach'.

One fun library that you can look at is easyAI, which is a simple Python engine for two-player games involving AI. With this framework, you can create and play various games such as Tic-Tac-Toe and Connect4. The steps for setting up and installing easyAI along with a user manual with examples are provided at zuko.github.io/easyAI/. To create a two-player game with easyAI, you need to simply create a sub-class of the TwoPlayersGame class (from the easyAI class). Then, you must define a set of methods that specify the nature of your game. To start the game, create an object of your game and then call the play method.

One very effective Natural Language and Text Processing library that Python offers is the NLTK, which is a platform for building Python programs that process data from human language. It basically provides the tools and libraries required to work with NLP. To make use of NLTK, you must first install it and then install the NLTK library. Optional installs such as NumPy are recommended, but not necessary. The steps and links for installation can be found at <http://www.nltk.org/install.html>. With the NLTK libraries, you can do many delightful things to a piece of text such as analyse the sentiments (positive or negative), tokenise (split the